

Danmarks
Tekniske
Universitet



Software Engineering Project Report 2

02161 Software Engineering I
DTU Compute, Spring 2026

Group 16

AUTHORS

Nicolai Søgaard -s234146
Nat Skovlund - s245203
Mathias Kjølbye - s240803
Vedanta Sinha - s245010
Viktor Munk - 240335

Git Repository:

<https://gitlab.gbar.dtu.dk/XXXX/software-project>
May 10, 2026

Contents

1	Introduction	1
1.1	Scope	1
1.2	Relation to Report 1	2
2	Implemented System Overview (Vedanta Sinha)	3
2.1	Implemented Functionality	3
2.2	Architecture Overview	3
2.3	Traceability to Report 1	3
3	Systematic Tests (White-Box Tests)	4
3.1	Choice of Methods	4
3.2	ProjectService.createProject(...)(Vedanta)	5
3.2.1	Method	5
3.2.2	Input Properties	6
3.2.3	Concrete Test Cases	6
3.2.4	Implementation of Tests	6
3.3	ActivityService.setActivityDetails(...)(Viktor)	7
3.3.1	Method	7
3.3.2	Input Properties	8
3.3.3	Concrete Test Cases	8
3.3.4	Implementation of Tests	9
3.4	ActivityService.addDeveloperToActivity(...)(Viktor)	9
3.4.1	Method	9
3.4.2	Input Properties	9
3.4.3	Concrete Test Cases	10
3.4.4	Implementation of Tests	10
3.5	TimeRegistrationService.registerTime(...)(Nat)	11
3.5.1	Method	11
3.5.2	Input Properties	12
3.5.3	Concrete Test Cases	12
3.5.4	Implementation of tests	12
3.6	AbsenceService.registerAbsence(...)	12
3.6.1	Method	12
3.6.2	Input Properties	13
3.6.3	Concrete Test Cases	13
3.7	Summary of White-Box Testing	13
4	Design by Contract ()	14
4.1	ProjectService.createProject(...)(Vedanta Sinha)	14
4.1.1	Method with Preconditions	14
4.1.2	Contract	15
4.1.3	Discussion	16

4.2	<code>ActivityService.setActivityDetails(...)</code> (Viktor)	16
4.2.1	Method with Assertions	16
4.2.2	Contract	17
4.2.3	Discussion	17
4.3	<code>ActivityService.addDeveloperToActivity(...)</code> (Viktor)	18
4.3.1	Method with Assertions	18
4.3.2	Contract	18
4.3.3	Discussion	19
4.4	<code>TimeRegistrationService.registerTime(Nat)</code>	19
4.4.1	Method with Assertions	19
4.4.2	Contract	21
4.4.3	Discussion	21
4.5	<code>AbsenceService.registerAbsence(...)</code>	22
4.5.1	Method with Assertions	22
4.5.2	Contract	22
4.6	Discussion of Contracts	22
5	Code Coverage (Viktor)	24
5.1	Measurement Setup	24
5.2	Results	24
5.3	Analysis by Layer	25
5.3.1	Service layer 77.25%	27
5.3.2	Domain layer 79.77%	27
5.4	Effect of the White-Box Tests	28
5.5	Limitations	29
5.6	Summary	29
6	Design Patterns and SOLID Principles (Nicolai)	30
6.1	Overall Design Choices	30
6.1.1	Domain Layer	31
6.1.2	Repository Layer	31
6.1.3	Service layer	32
6.1.4	User Interface Layer	33
6.1.5	Overall Design Considerations	33
6.2	SOLID-principles	33
6.3	Single Responsible Principle	34
6.3.1	Domain-classes	34
6.3.2	Repositories	34
6.3.3	Service layer	35
6.4	Dependency Inversion Principle	35
6.5	Other SOLID-Principles	37
6.5.1	OCP, Open/Close principle	37
6.5.2	Interface Segregation Principle	37
6.5.3	Liskov Substitution Principle	38

6.5.4	Final remarks	38
7	Conclusion	39
7.1	Reflection on the Development Process	40
8	Collaboration and Contribution Weights	40
9	Appendix	41
9.1	Git Repository	41
9.2	Run Instructions	41
	List of Figures	42
	List of Tables	43
	References	44

1 Introduction

This report documents the implementation and evaluation of our software project planner. The system was developed to support the main workflow of a project management environment: creating projects, creating and planning activities, assigning developers, registering work hours, handling absence, assigning project leaders, and generating project status information. The purpose of the system is to make project planning and follow-up more structured, while keeping the core business logic clear and testable.

Report 1 focused on requirements, use cases, domain understanding, and the initial design of the system. This report continues from that foundation and evaluates the implemented solution. The focus is therefore not only on what the system is intended to do, but also on how the implementation supports the requirements in practice. To keep this connection clear, the report follows the same central flow throughout: first describing the implemented system, then testing selected behaviour systematically, then specifying contracts for important methods, and finally evaluating the design and coverage of the final product.

The implementation is organised into separate layers. The domain layer contains the core objects such as projects, activities, developers, absences, and time registrations. The repository layer stores and retrieves these objects, while the service layer contains the main business logic. The user interface layer provides access to the functionality through the JavaFX interface and console interface. This structure makes it possible to test the core logic independently from the user interface, which is important because the most critical rules are located in the service layer.

The report evaluates the system through several complementary techniques. Systematic white-box tests are used to analyse selected methods in detail and derive concrete test cases from their input properties and control flow. Design by Contract is then used on the same methods to express their expected behaviour as preconditions and postconditions. Code coverage is used to measure how much of the implementation is exercised by the tests, and the design section discusses how the final code structure relates to design principles such as separation of concerns, dependency inversion, and single responsibility.

Together, these sections provide a coherent assessment of the final system. The report shows how the implementation is connected to the original design, how the most important behaviour is tested, and how the chosen architecture supports maintainability and further development.

1.1 Scope

The main emphasis of this project is placed on the core service layer functionality. This includes the prior mentioned project and activity management, resource allocation, progress tracking, and reporting. This is the most relevant part for detailed analysis, since it contains the central application logic. We also worked on creating a clean and professional graphical

user interface (GUI) to realistically expedite the planned functionality of the system.

1.2 Relation to Report 1

This report builds on the analysis and design presented in Report 1. However, where Report 1 focused on requirements, use cases, and system design, this report focuses on the implemented solution and its evaluation.

During the development some implementation details were adjusted to keep the scope manageable, and to fulfill the requirement. The final system still reflects the central ideas and requirements identified in the earlier report.

2 Implemented System Overview (Vedanta Sinha)

This section gives an overview of the implemented system and structure. It focuses on the structure and functionality of the system. This section further aims to explain what the system does, how it does it, and how the structure of our code helps fulfill the required functionality.

2.1 Implemented Functionality

All the functionality of this system aims to help a company assign and finish projects in an orderly manner. The main functionalities include the creation of a project, creation and planning of activities, assignment of developers to activities, time registration on activities, absence registration, assignment of project leaders, project reporting, and progress tracking.

2.2 Architecture Overview

The system's architecture is made up from multiple layers, which are separated from each other logically based on their responsibilities. This makes them easier to test and scale.

These layers consist of the domain layer, which holds the core entities of the software, such as Projects, Activities, Developers, Absences, and Time Registration. Above the domain layer, one can find the repository layer, which functions as the repository. Its functionality is to manage data storage and retrieval. This specific iteration of the repository layer uses 'in-memory', which has its upside: easier coding, faster creation, retrieval, and storage of objects, since the memory operates in the RAM. It also has its downsides, since the repository will be cleared when the system boots off.

Building on top of the repository level's ability to store and retrieve objects, there is the service level. This is the layer tasked with applying logic and coordinating operations together so that the system can finish the tasks required. It is also tasked with ensuring that inputs are valid and consistent, and a real world company would use this layer to ensure that its employees couldn't break company policies.

On top of this is the User Interface, which is the interface that the user of the software uses in order to facilitate their commands. For this, there is a Graphical user interface being used, so that it is easiest for normal people in a company to use the software.

2.3 Traceability to Report 1

The system that is currently implemented is directly based on the the analysis done during Report 1, where sections such as the design and requirements of the software were refined. Overall, the implementation maintains consistency with the specifications and system design established in Report 1.

3 Systematic Tests (White-Box Tests)

In this section, we present systematic white-box tests for selected methods in the implemented system. Each group member selected one action-oriented method, as required by the project brief. Getter and setter methods were not selected, because they do not contain enough behaviour, branching, or validation logic to justify systematic white-box testing.

The test design follows the two table template from the course lectures. The first table identifies the relevant input properties for a method, such as valid and invalid values, boundary values, missing objects, and state combinations. The second table turns those properties into concrete test cases with specific input values and expected results. Each row in the concrete test table corresponds to an executable JUnit or Cucumber test.

The white-box tests complement the BDD tests. The Cucumber scenarios verify that the system supports the required user-level behavior, while the white-box tests focus on implementation-level paths inside specific methods. This is useful because the service layer contains the most important business rules, including validation, repository lookups, filtering, and state changes.

3.1 Choice of Methods

The following methods were selected:

- `ProjectService.createProject(...)`
- `ActivityService.createActivity(...)`
- `ActivityService.addDeveloperToActivity(...)`
- `TimeRegistrationService.registerTime(...)`
- `AbsenceService.registerAbsence(...)`

These methods were chosen because they represent key system behaviour and contain validation logic, state changes, or coordination across multiple classes.

3.2 ProjectService.createProject(...)(Vedanta)

3.2.1 Method

This is the most crucial method, as without a project, developers cannot be assigned, they cannot make reports, and activities cannot be given. This method is responsible for creating new projects and working with the Project Repository, storing the project.

The method checks whether the provided name is null or empty. If the input is invalid, it returns with an error. This is done to make sure that only valid projects are created.

If the input is valid, a new object 'project' is created with an ID and the given name. Finally, the project is stored in the repository, and the created object is returned. This process ensures that all projects are uniquely identifiable and consistently stored.

```

1      public Project createProject(String name) { // Only allows
           strings
2          if (name == null) {
3              throw new IllegalArgumentException("Project_
                   name_cannot_be_empty");
4          }
5          boolean hasContent = false;
6          for (int i = 0; i < name.length(); i++) {
7              if (name.charAt(i) != ' ') {
8                  hasContent = true;
9              }
10         }
11         if (!hasContent) {
12             throw new IllegalArgumentException("Project_
                   name_cannot_be_empty");
13         }
14         String id = projectRepository.generateProjectId();
15         Project project = new Project(id, name);
16         projectRepository.add(project);
17         return project;
18     }

```

3.2.2 Input Properties

Property	Description
Project name valid	Non-empty project name
Project name invalid	Empty or blank project name

Table 1: Input properties for createProject(...)

Table 1 shows how the method is based on its property of defining validity and invalidity in the inputs given. A valid project name must not be empty; it can even be named after another project, since the ID given will be different. Invalid inputs include null values or blank strings, which do not provide useful information and could lead to ambiguity in the system.

3.2.3 Concrete Test Cases

Test case	Input	Expected result
TC1 (Tests 5)	"WebShop"	Project is created successfully
TC2 (Tests 6)	" "	Exception: project name cannot be empty

Table 2: Concrete test cases for createProject(...)

To check the correctness of the method, concrete test cases have been made, covering both valid and invalid scenarios.

When a valid project name, such as "WebShop" is provided, the system successfully creates and stores a new project. On the other hand, when an empty string is used as input, the method throws an exception

```

1 [Results of Cucumber tests]
2
3 %TESTS 5,Create a project
4
5 %TESTE 5,Create a project
6
7 %TESTS 6,Cannot create a project with an empty name
8
9 %TESTE 6,Cannot create a project with an empty name

```

3.2.4 Implementation of Tests

The planned test cases were implemented as executable Cucumber tests in the project.

3.3 ActivityService.setActivityDetails(...) (Viktor)

These tests verify both valid and invalid input behaviour.

TESTS validates that the test has started (TEST START), and TESTE validates that the test has ended (TEST ENDED). Test 5 shows that a Project with a valid name is created successfully. On the other hand, test 6 shows that a Project with an invalid name fails to be produced, and instead gives an error.

3.3 ActivityService.setActivityDetails(...) (Viktor)

3.3.1 Method

```
1 public void setActivityDetails(String projectId, String activityName,
2                               double budgetedHours,
3                               int startWeek, int startYear,
4                               int endWeek, int endYear) {
5     if (budgetedHours < 0) {
6         throw new IllegalArgumentException(
7             "Budgeted hours cannot be negative");
8     }
9     if (startWeek != 0 && (startWeek < 1 || startWeek > 53)) {
10         throw new IllegalArgumentException(
11             "Start week must be between 1 and 53");
12     }
13     if (endWeek != 0 && (endWeek < 1 || endWeek > 53)) {
14         throw new IllegalArgumentException(
15             "End week must be between 1 and 53");
16     }
17     if (startWeek != 0 && endWeek != 0) {
18         int start = startYear * 100 + startWeek;
19         int end = endYear * 100 + endWeek;
20         if (start > end) {
21             throw new IllegalArgumentException(
22                 "Start week must be before or equal to end week");
23         }
24     }
25     Activity activity = getActivity(projectId, activityName);
26     activity.setBudgetedHours(budgetedHours);
27     activity.setStartWeek(startWeek);
28     activity.setStartYear(startYear);
29     activity.setEndWeek(endWeek);
30     activity.setEndYear(endYear);
31 }
```

This method was chosen because it contains multiple independent validation branches (budget, start week range, end week range, and period ordering), making it well-suited for equivalence-class partitioning and boundary-value analysis. Getter and setter methods were deliberately excluded per the project brief.

3.3.2 Input Properties

Property	Description
Budget valid	$\text{budgetedHours} \geq 0$
Budget invalid	$\text{budgetedHours} < 0$
Start week unset	$\text{startWeek} = 0$ (no constraint applied)
Start week valid	$1 \leq \text{startWeek} \leq 53$
Start week invalid	$\text{startWeek} < 1$ or > 53
End week unset	$\text{endWeek} = 0$
End week valid	$1 \leq \text{endWeek} \leq 53$
End week invalid	$\text{endWeek} < 1$ or > 53
Period valid	$\text{start} \leq \text{end}$ (encoded as $\text{year} \times 100 + \text{week}$)
Period invalid	$\text{start} > \text{end}$

Table 3: Input properties for setActivityDetails(...)

3.3.3 Concrete Test Cases

TC	budgetedHours	startWeek / endWeek	Expected result
TC1	100.0	10 / 12 (2026)	Details set successfully
TC2	-1.0	10 / 12 (2026)	Exception: hours cannot be negative
TC3	0.0	0 / 0	Details set (unset weeks accepted)
TC4	50.0	1 / 53 (2026)	Details set (boundary weeks)
TC5	50.0	54 / 55 (2026)	Exception: start week out of range
TC6	50.0	10 / 54 (2026)	Exception: end week out of range
TC7	50.0	12 / 10 (2026)	Exception: start must be before end
TC8	50.0	5 (2026) / 3 (2025)	Exception: start must be before end

Table 4: Concrete test cases for setActivityDetails(...)

TC4 targets the boundary values 1 and 53 for both week parameters. TC8 verifies that the year is included in the ordering comparison: week 5 of 2026 as start and week 3 of 2025 as end raises an exception because $2026 \times 100 + 5 > 2025 \times 100 + 3$.

3.4 ActivityService.addDeveloperToActivity(...)(Viktor)

3.3.4 Implementation of Tests

The test cases above are implemented as JUnit 5 tests in `ActivityServiceTest.java`. Each case either asserts a successful state change on the activity object or verifies that the correct `IllegalArgumentException` message is thrown.

3.4 ActivityService.addDeveloperToActivity(...)(Viktor)

3.4.1 Method

```
1 public void addDeveloperToActivity(String projectId,
2                                   String activityName,
3                                   String developerInitials) {
4     Activity activity = getActivity(projectId, activityName);
5
6     Optional<Developer> result =
7         developerRepository.findByInitials(developerInitials);
8     if (!result.isPresent()) {
9         throw new IllegalArgumentException(
10             "Developer not found: " + developerInitials);
11     }
12     Developer developer = result.get();
13
14     activity.addDeveloper(developer);
15 }
```

3.4.2 Input Properties

Property	Description
Project exists	The project ID refers to an existing project in the repository.
Project does not exist	The project ID does not match any project in the repository.
Activity exists	The activity name refers to an existing activity in the selected project.
Activity does not exist	The activity name does not match an activity in the selected project.
Developer exists	The developer initials match a developer in the developer repository.
Developer does not exist	The developer initials are unknown to the system.
Developer already assigned	The developer is already assigned to the activity. The domain method prevents duplicate entries.

Table 5: Input properties for `addDeveloperToActivity(...)`

3.4 ActivityService.addDeveloperToActivity(...)(Viktor)

These properties cover both the successful path and the relevant error paths in the method. If the project or activity cannot be found, the method fails through getActivity(...). If the developer initials are unknown, the method throws an IllegalArgumentException. If all lookups succeed, the method performs the assignment.

3.4.3 Concrete Test Cases

Test case	Input and system state	Expected result
TC1	Existing project, activity Requirements exists, developer huba exists.	Developer huba is assigned to activity Requirements.
TC2	Existing project, activity Requirements exists, developer initials zzzz do not exist.	IllegalArgumentException: Developer not found: zzzz.
TC3	Existing project, activity name Design does not exist, developer huba exists.	IllegalArgumentException: Activity not found: Design.
TC4	Project ID 99999 does not exist, activity Requirements is requested, developer huba exists.	IllegalArgumentException because the activity cannot be found for the project.
TC5	Existing project, activity Requirements exists, huba is added twice.	huba appears only once in the activity's assigned developer list.

Table 6: Concrete test cases for addDeveloperToActivity(...)

Together, the test cases exercise the method's main decision points: activity lookup, developer lookup, and the final state change on the Activity object. This makes the tests systematic rather than only example-based.

3.4.4 Implementation of Tests

The test cases are represented by the existing Cucumber scenarios for assigning developers to activities. The feature file contains a successful case where developer huba is added to the activity Requirements, and an error case where the unknown initials zzzz are rejected. These scenarios test the two most important branches in the method: successful lookup and assignment, and failed developer lookup.

The method is suitable for white-box testing because the result depends on internal repository lookups and on the state change performed on the Activity object. If the project or activity cannot be found, getActivity(...) throws an exception. If the developer cannot be found, addDeveloperToActivity(...) throws an exception. If both lookups succeed, the developer is added to the activity.

3.5 TimeRegistrationService.registerTime(...)(Nat)

3.5.1 Method

```

1  public TimeRegistration registerTime(String developerInitials,
2      String projectId, String activityName, LocalDate date, double
3      hours) {
4      if (hours <= 0 || hours % 0.5 != 0) {
5          throw new IllegalArgumentException("Hours must be a
6              positive multiple of 0.5");
7      }
8      // Pre-conditions (hold after defensive validation above)
9      assert developerInitials != null : "developerInitials must
10         not be null";
11      assert projectId != null : "projectId must not be null";
12      assert activityName != null : "activityName must not be null"
13         ;
14      assert date != null : "date must not be null";
15      assert hours > 0 : "hours must be positive";
16      assert hours % 0.5 == 0 : "hours must be a multiple of 0.5";
17
18      Developer developer = developerRepository.findByInitials(
19          developerInitials)
20          .orElseThrow(() -> new IllegalArgumentException("
21              Developer not found: " + developerInitials));
22
23      Activity activity = projectRepository.findActivity(projectId,
24          activityName)
25          .orElseThrow(() -> new IllegalArgumentException("
26              Activity not found: " + activityName));
27
28      TimeRegistration registration = new TimeRegistration(
29          developer, activity, date, hours);
30      activity.addTimeRegistration(registration);
31
32      // Post-conditions
33      assert activity.getTimeRegistrations().contains(registration)
34         : "registration must be stored on activity";
35      assert registration.getHours() == hours : "registered hours
36         must match input";
37
38      return registration;
39  }

```

3.6 AbsenceService.registerAbsence(...)

3.5.2 Input Properties

Property	Description
Hours valid	Positive number and multiple of 0.5
Hours zero or negative	Invalid number of hours
Hours not multiple of 0.5	Invalid granularity
Activity exists	The activity exists in the specified project

Table 7: Input properties for `registerTime(...)`

3.5.3 Concrete Test Cases

Test case	Input	Expected result
TC1 (Tests 11)	2.5 hours	Time registration stored successfully
TC2 (Tests 12)	0.0 hours	Exception: invalid number of hours
TC3 (Tests 13)	1.3 hours	Exception: must be multiple of 0.5

Table 8: Concrete test cases for `registerTime(...)`

3.5.4 Implementation of tests

The test cases are designed to test all the possible scenarios that the method has to handle.

```
1 [Results of cucumber tests ]
2 %TESTS 11,Register time on an activity
3
4 %TESTE 11,Register time on an activity
5 %TESTS 12,Cannot register zero hours on an activity
6
7 %TESTE 12,Cannot register zero hours on an activity
8
9 %TESTS 13,Cannot register hours not a multiple of 0.5
10
11 %TESTE 13,Cannot register hours not a multiple of 0.5
```

3.6 AbsenceService.registerAbsence(...)

3.6.1 Method

[Insert method code here]

3.6.2 Input Properties

Property	Description
Developer is free in period	Absence can be registered
Developer is busy in period	Absence must be rejected
Valid week interval	Start and end period form a meaningful interval

Table 9: Input properties for `registerAbsence(...)`

3.6.3 Concrete Test Cases

Test case	Input	Expected result
TC1	Vacation in free period	Absence registered successfully
TC2	Vacation in already booked period	Exception: vacation denied

Table 10: Concrete test cases for `registerAbsence(...)`

3.7 Summary of White-Box Testing

The systematic tests were derived from input partitions and boundary-relevant cases. The goal was to ensure that each selected method was tested for both normal behaviour and relevant error conditions. These white-box tests complement the existing BDD tests and provide more explicit validation of implementation logic.

4 Design by Contract ()

Design by Contract is used to describe the expected behaviour of selected methods in a precise and systematic way. For each chosen method, the contract defines the conditions that must hold before the method is executed, called preconditions, and the conditions that must hold after the method has completed successfully, called postconditions.

In this project, Design by Contract is applied to the same methods that were analysed in the white-box testing section. This creates a clear connection between the test cases and the formal expectations of the implementation. The white-box tests show concrete examples of valid and invalid behaviour, while the contracts generalise this behaviour into logical rules.

The contracts are expressed both as mathematical formulas and, where relevant, as Java `assert` statements. The formulas make the intended behaviour clear in the report, while the assert statements allow important assumptions and guarantees to be checked at runtime when assertions are enabled with `-ea`. This helps document the service layer logic and supports testing of the most important business rules in the system.

4.1 ProjectService.createProject(...)(Vedanta Sinha)

4.1.1 Method with Preconditions

The method requires certain conditions to validate an input; these are the preconditions that must be met. However, since there are many different types of valid input, it is cleaner to instead implement a system that flags invalid inputs.

There is also a requirement for systematic postconditioning. This is done to ensure that other parts of the system can work fluidly with the project creation system.

4.1 ProjectService.createProject(...)(Vedanta Sinha)

```
1      public Project createProject(String name) { // Only allows
           strings
2          if (name == null) {
3              throw new IllegalArgumentException("Project_
                   name_cannot_be_empty");
4          }
5          boolean hasContent = false;
6          for (int i = 0; i < name.length(); i++) {
7              if (name.charAt(i) != ' ') {
8                  hasContent = true;
9              }
10         }
11         if (!hasContent) {
12             throw new IllegalArgumentException("Project_
                   name_cannot_be_empty");
13         }
14         String id = projectRepository.generateProjectId();
15         Project project = new Project(id, name);
16         projectRepository.add(project);
17         return project;
18     }
```

4.1.2 Contract

Precondition:

The preconditions of this method can be found from lines 2 to 12. The first of these is "name == null"; this simple line ensures that if the creator of the project has forgotten to insert anything in the name section, it will throw him or her an error.

$$name \neq null$$

Next, the system must ensure a project cannot be named something like " ". For this, the system takes the default boolean, true or false, of `hasContent = False`, meaning that by default, if an input is unable to convince the system that it has a valid name, it will be thrown out as an error. Now, using a loop, the system looks at every character in the input, and if any character is not " " (space). If it finds a legitimate character, it updates to `hasContent = True`.

$$name.trim().length() > 0$$

4.2 ActivityService.setActivityDetails(...) (Viktor)

Postcondition: Line 14 is where the actual formatting of the Project name happens; this has to be done correctly, or else the other modules will be unable to retrieve projects. In a way, these postconditions will be the precondition for the other modules. It starts with creating an ID for the project, using `ProjectRepository.generateProjectID()`. This gives the project a unique ID, so even if two projects are named the same thing, they can still be differentiated. The name and ID are kept in a tuple `Project(id, name);`, and is present in the `ProjectRepository`.

4.1.3 Discussion

The use of preconditions and postconditions provides a clear structure for this and other methods to follow and rely upon. The precondition ensures that only valid project names are accepted. The postcondition guarantees that a project object is created and stored consistently, so that other methods can use them properly.

4.2 ActivityService.setActivityDetails(...) (Viktor)

4.2.1 Method with Assertions

```
1 public void setActivityDetails(String projectId, String activityName,
2                               double budgetedHours,
3                               int startWeek, int startYear,
4                               int endWeek,  int endYear) {
5     if (budgetedHours < 0) {
6         throw new IllegalArgumentException(
7             "Budgeted hours cannot be negative");
8     }
9     if (startWeek != 0 && (startWeek < 1 || startWeek > 53)) {
10        throw new IllegalArgumentException(
11            "Start week must be between 1 and 53");
12    }
13    if (endWeek != 0 && (endWeek < 1 || endWeek > 53)) {
14        throw new IllegalArgumentException(
15            "End week must be between 1 and 53");
16    }
17    // check that start is not after end
18    if (startWeek != 0 && endWeek != 0) {
19        int start = startYear * 100 + startWeek;
20        int end   = endYear   * 100 + endWeek;
21        if (start > end) {
22            throw new IllegalArgumentException(
23                "Start week must be before or equal to end week");
24        }
25    }
26
27    assert budgetedHours >= 0
```

4.2 ActivityService.setActivityDetails(...) (Viktor)

```
28         : "budgetedHours_must_be_non-negative";
29     assert startWeek == 0 || (startWeek >= 1 && startWeek <= 53)
30         : "startWeek_must_be_0_or_between_1_and_53";
31     assert endWeek == 0 || (endWeek >= 1 && endWeek <= 53)
32         : "endWeek_must_be_0_or_between_1_and_53";
33
34     Activity activity = getActivity(projectId, activityName);
35     activity.setBudgetedHours(budgetedHours);
36     activity.setStartWeek(startWeek);
37     activity.setStartYear(startYear);
38     activity.setEndWeek(endWeek);
39     activity.setEndYear(endYear);
40
41     assert activity.getBudgetedHours() == budgetedHours
42         : "budgeted_hours_must_match_input";
43     assert activity.getStartWeek() == startWeek
44         : "start_week_must_match_input";
45     assert activity.getEndWeek() == endWeek
46         : "end_week_must_match_input";
47 }
```

4.2.2 Contract

Precondition:

$budgetedHours \geq 0 \wedge (startWeek = 0 \vee 1 \leq startWeek \leq 53) \wedge (endWeek = 0 \vee 1 \leq endWeek \leq 53)$

$\wedge (startWeek \neq 0 \wedge endWeek \neq 0) \Rightarrow startYear \times 100 + startWeek \leq endYear \times 100 + endWeek$

Postcondition:

$activity.getBudgetedHours() = budgetedHours$
 $\wedge activity.getStartWeek() = startWeek$
 $\wedge activity.getEndWeek() = endWeek$

4.2.3 Discussion

The precondition captures the four independent constraints that must hold before the method may safely mutate the activity, non negative budget, both week parameters either unset (0) or within the ISO week number range 1–53, and a chronological ordering of the period when both endpoints are provided. The value 0 serves as a sentinel meaning “not yet planned”, which is an explicit requirement from the project description.

The postcondition guarantees that all three planning fields on the activity object reflect the supplied arguments after a successful invocation. The assert-statements in the code enforce both conditions at runtime when the JVM is started with `-ea`.

4.3 ActivityService.addDeveloperToActivity(...)(Viktor)

4.3.1 Method with Assertions

```

1 public void addDeveloperToActivity(String projectId,
2                                   String activityName,
3                                   String developerInitials) {
4     assert projectId != null : "projectId_must_not_be_null";
5     assert activityName != null : "activityName_must_not_be_null";
6     assert developerInitials != null
7         : "developerInitials_must_not_be_null";
8
9     Activity activity = getActivity(projectId, activityName);
10    int assignedBefore = activity.getAssignedDevelopers().size();
11
12    Optional<Developer> result =
13        developerRepository.findByInitials(developerInitials);
14    if (!result.isPresent()) {
15        throw new IllegalArgumentException(
16            "Developer_not_found:" + developerInitials);
17    }
18    Developer developer = result.get();
19
20    activity.addDeveloper(developer);
21
22    assert activity.getAssignedDevelopers().contains(developer)
23        : "developer_must_be_assigned_to_the_activity";
24    assert activity.getAssignedDevelopers().size() >= assignedBefore
25        : "assignment_must_not_remove_developers";
26 }

```

4.3.2 Contract

Precondition:

$$projectId \neq null \wedge activityName \neq null \wedge developerInitials \neq null$$

$$projectExists(projectId) \wedge activityExists(projectId, activityName) \wedge developerExists(developerInitials)$$

Postcondition:

$$developer \in activity.getAssignedDevelopers()$$

$$activity.getAssignedDevelopers()_{after}.size() \geq activity.getAssignedDevelopers()_{before}.size()$$

4.4 TimeRegistrationService.registerTime(Nat)

activity = projectRepository.findActivity(projectId, activityName)

developer = developerRepository.findByInitials(developerInitials)

4.3.3 Discussion

The precondition captures the constraints that must hold before the method may safely assign a developer to an activity. The three string parameters must not be null, the project must exist, the activity must exist inside that project, and the developer initials must refer to a known developer in the system. These conditions match the purpose of the method: it should only connect already existing domain objects, not create new projects, activities, or developers.

The postcondition guarantees the state change caused by a successful invocation. After the method has run without throwing an exception, the selected developer is contained in the activity's assigned developer list. The size condition states that the operation must not remove existing developers from the activity. If the developer was not already assigned, the list grows by one. If the developer was already assigned, the Activity class prevents duplicates, so the list may keep the same size while still satisfying the contract.

The assert statements in the code express the same idea at runtime when the JVM is started with `-ea`. The precondition assertions document the required inputs, while the postcondition assertions check that the assignment has actually been stored on the activity. This makes the contract useful both as documentation and as a runtime check during testing.

4.4 TimeRegistrationService.registerTime(Nat)

4.4.1 Method with Assertions

Listing 1: Register Time Method

```
1 public TimeRegistration registerTime(String developerInitials, String
   projectId, String activityName, LocalDate date, double hours) {
2     if (hours <= 0 || hours % 0.5 != 0) {
3         throw new IllegalArgumentException("Hours must be a
           positive multiple of 0.5");
4     }
5
6     // Pre-conditions (hold after defensive validation above)
7     assert developerInitials != null : "developerInitials must
           not be null";
8     assert projectId != null : "projectId must not be null";
9     assert activityName != null : "activityName must not be null"
       ;
```

4.4 TimeRegistrationService.registerTime(Nat)

```
10  assert date != null : "date_must_not_be_null";
11  assert hours > 0 : "hours_must_be_positive";
12  assert hours % 0.5 == 0 : "hours_must_be_a_multiple_of_0.5";
13
14  Developer developer = developerRepository.findByInitials(
15      developerInitials)
16      .orElseThrow(() -> new IllegalArgumentException("
17          Developer_not_found:_" + developerInitials));
18
19  Activity activity = projectRepository.findActivity(projectId,
20      activityName)
21      .orElseThrow(() -> new IllegalArgumentException("
22          Activity_not_found:_" + activityName));
23
24  TimeRegistration registration = new TimeRegistration(
25      developer, activity, date, hours);
26  activity.addTimeRegistration(registration);
27
28  // Post-conditions
29  assert activity.getTimeRegistrations().contains(registration)
30      : "registration_must_be_stored_on_activity";
31  assert registration.getHours() == hours : "registered_hours_
32      must_match_input";
33
34  return registration;
35 }
36
37 public void editTimeRegistration(String developerInitials, String
38     projectId, String activityName, LocalDate date, double
39     newHours) {
40     if (newHours <= 0 || newHours % 0.5 != 0) {
41         throw new IllegalArgumentException("Hours_must_be_a_
42             positive_multiple_of_0.5");
43     }
44
45     Developer developer = developerRepository.findByInitials(
46         developerInitials)
47         .orElseThrow(() -> new IllegalArgumentException("
48             Developer_not_found:_" + developerInitials));
49
50     Activity activity = projectRepository.findActivity(projectId,
51         activityName)
52         .orElseThrow(() -> new IllegalArgumentException("
53             Activity_not_found:_" + activityName));
```


4.4 TimeRegistrationService.registerTime(Nat)

```
41     List<TimeRegistration> registration = activity.  
        getTimeRegistrations();  
42     for (TimeRegistration timeRegistration : registration) {  
43         if (timeRegistration.getDeveloper().equals(developer)  
            && timeRegistration.getDate().equals(date)){  
44             timeRegistration.setHours(newHours);  
45         }  
46         else{  
47             throw new IllegalArgumentException("No time  
            registration found for that date");  
48         }  
49     }  
50 }  
51 }  
52 }
```

4.4.2 Contract

Precondition:

$$hours > 0 \wedge hours \bmod 0.5 = 0$$

$$projectExists(projectId) \wedge activityExists(projectId, activityName) \wedge developerExists(initials) \\ developer \in activity.assignedDevelopers$$

Postcondition:

$$\exists r \in activity.timeRegistrations : r.hours = hours$$

4.4.3 Discussion

The precondition ensures that only valid amounts of time can be added to existing activities that the developer is added to. The postcondition ensures that the time has been registered correctly.

4.5 AbsenceService.registerAbsence(...)

4.5.1 Method with Assertions

Listing 2: Register Absence Method

```

1  public Absence registerAbsence(String developerInitials,
2                                Absence.Type type,
3                                int startWeek, int startYear,
4                                int endWeek, int endYear) {
5
6      Developer developer = developerRepository.findByInitials(
7          developerInitials)
8          .orElseThrow(() -> new IllegalArgumentException(
9              "Developer not found: " + developerInitials));
10
11     if (type == Absence.Type.VACATION &&
12         isDeveloperBusyInPeriod(developer, startWeek, startYear,
13                                 endWeek, endYear)) {
14
15         throw new IllegalArgumentException(
16             "Vacation denied: developer is assigned to an
17             activity in this period");
18     }
19
20     Absence absence = new Absence(developer, type,
21                                   startWeek, startYear,
22                                   endWeek, endYear);
23
24     absenceRepository.add(absence);
25
26     return absence;
27 }

```

4.5.2 Contract

Precondition:

$developerExists(initials) \wedge periodIsValid(startWeek, startYear, endWeek, endYear)$

Postcondition:

$absenceRegistered \vee exceptionThrown$

4.6 Discussion of Contracts

The contracts make assumptions and guarantees explicit. They improve readability, support debugging, and document the intended behaviour of the methods. In our implementation,

assertions were added to enforce preconditions and postconditions directly in code.

5 Code Coverage (Viktor)

Code coverage is a quantitative metric that expresses the proportion of production code exercised at least once during a test suite execution. As Edsger W. Dijkstra famously noted, “testing can only reveal the presence of bugs, never their absence” coverage does not resolve this fundamental limitation, but it transforms an otherwise opaque question into a concrete, measurable property. A low coverage figure therefore guarantees that some code is untested, while a high figure is a necessary but not sufficient condition for confidence in the suite.

In the context of this project, the analysis serves two concrete purposes: to validate that the TDD workflow was genuinely followed in the service layer, and to document the concrete effect of adding the systematic white-box tests from Section 3.

5.1 Measurement Setup

Coverage was collected using Visual Studio Code’s built in “Run Tests with Coverage” action in the Test Explorer. VS Code’s Java Test Runner extension bundles the JaCoCo agent and attaches it to the JVM at test time, so no JaCoCo entry is required in `pom.xml`. JaCoCo performs on-the-fly bytecode instrumentation, probes are inserted when each class is loaded, recording which instructions and branches are executed. The results appear in VS Code’s `test coverage` panel.

The suite that underpins the measurement consists of 57 passing tests: 32 Cucumber BDD scenarios covering 12 use cases, and 25 JUnit white-box tests, 14 in `ProjectServiceTest`, 11 in `ActivityServiceTest` designed using equivalence partitioning and boundary value analysis.

5.2 Results

Table 11 summarises the per-package figures. The overall weighted total of 29% is dominated by the UI layer, which accounts for 880 of the 1245 executable lines and has 0% coverage by design (discussed in Section 5.3).

Package	Line coverage	Lines
<code>com.planner.domain</code>	79.77%	105
<code>com.planner.repository</code>	89.16%	53
<code>com.planner.service</code>	78.31%	207
<code>com.planner.ui</code>	0.00%	880
Total (weighted)	29.00%	1245

Table 11: Line coverage per package as reported by JaCoCo via VS Code’s `test coverage` panel.

Figure 1 shows the top-level `test coverage` panel after running all 57 tests. The panel

is structured hierarchically from the `com.planner` root down through sub packages, only the package level is shown here, with the per class views presented in Section 5.3.

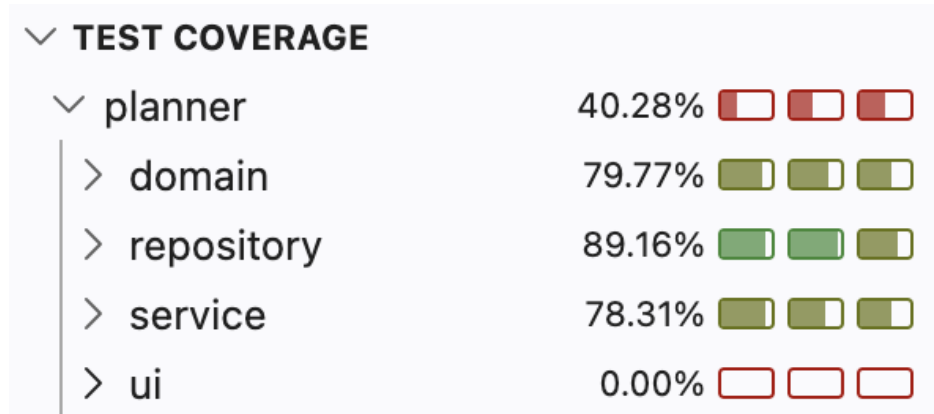


Figure 1: VS Code test coverage panel at package level after executing all 57 tests.

5.3 Analysis by Layer

Repository Layer

Figure 2 shows the per-class breakdown of the repository layer. `ProjectRepository.java` achieves 100% line coverage, as it is reached on every Cucumber scenario that creates or looks up a project. `AbsenceRepository.java` reaches 94.44%, with the small gap stemming from a defensive branch that handles a missing or empty absence list. `DeveloperRepository.java` sits at 79.49%; the uncovered lines are in the `loadFromFile` fallback that inserts `huba` when the resource file cannot be read, a branch that is unreachable under normal test conditions and is therefore intentionally left unexercised.

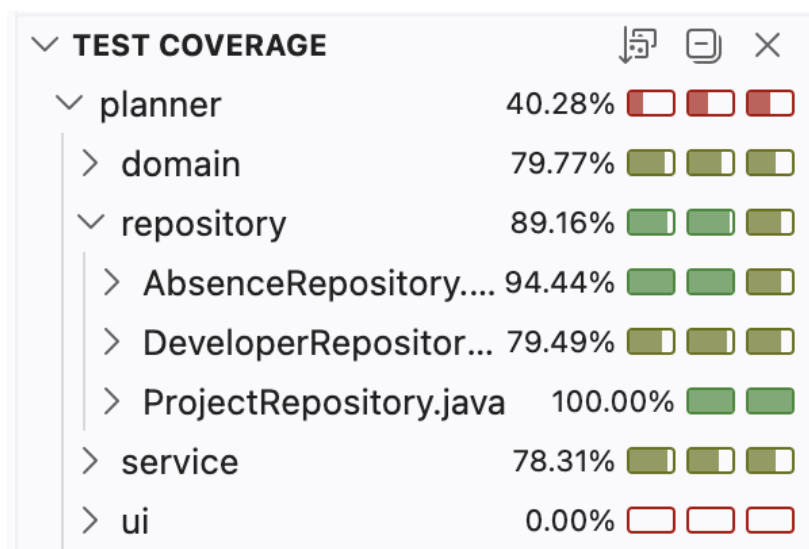


Figure 2: Per-class coverage within `com.planner.repository`.

UI Layer

Figure 3 shows the per-class breakdown of the UI layer. All four classes — `App.java`, `AppContext.java`, `ConsoleUI.java`, and `MainWindow.java` — report 0.00% coverage. This is a deliberate consequence of the project’s testing strategy: the project brief explicitly states that the user interface does not need to be developed test-first, and Robert C. Martin’s argument that the presentation layer sits at the edge of the system applies directly here. Every meaningful action in the UI delegates immediately to a service-layer method that is already covered by the Cucumber and JUnit test suites. The UI layer is therefore tested manually through the demonstration on 11 May rather than through automated tests.

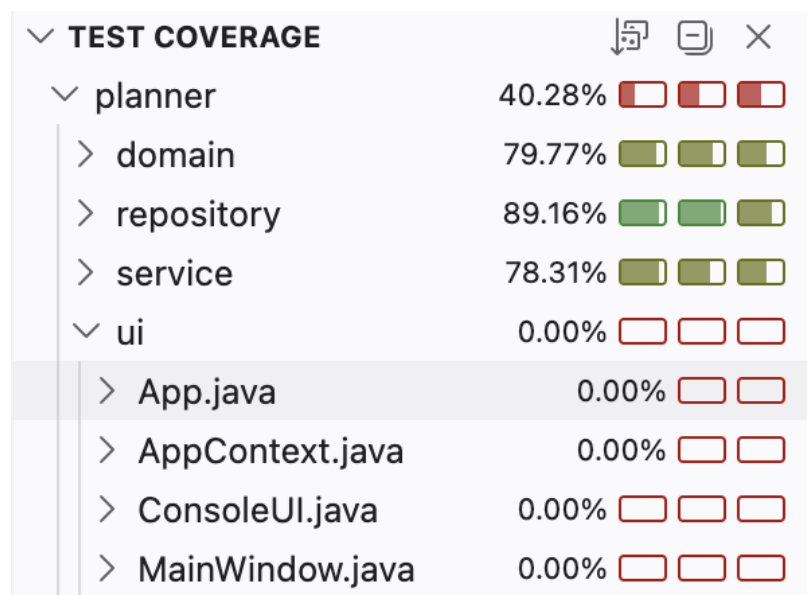


Figure 3: Per-class coverage within `com.planner.ui`.

5.3.1 Service layer 77.25%

This layer was developed strictly test first, so every public method in the six services has at least one Cucumber scenario. Figure 4 shows the per class breakdown. The 25 JUnit tests raise branch coverage further on `ProjectService` (85.25%) and `ActivityService` (82.18%) by targeting boundary values and edge cases, null/empty/whitespace names, week numbers 0, 1, 53, 54 that the BDD scenarios do not reach. `DeveloperService` sits notably lower at 23.08% because the project brief specifies that developers are imported from the HR file rather than created through the system, leaving most of its methods unused in any scenario.

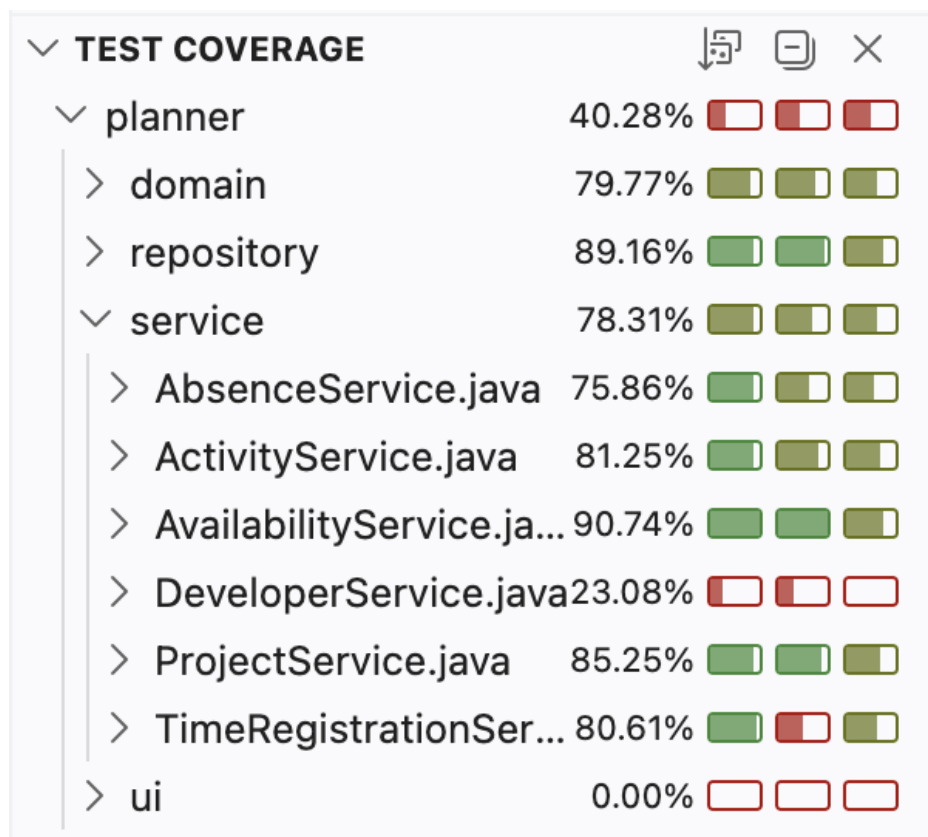
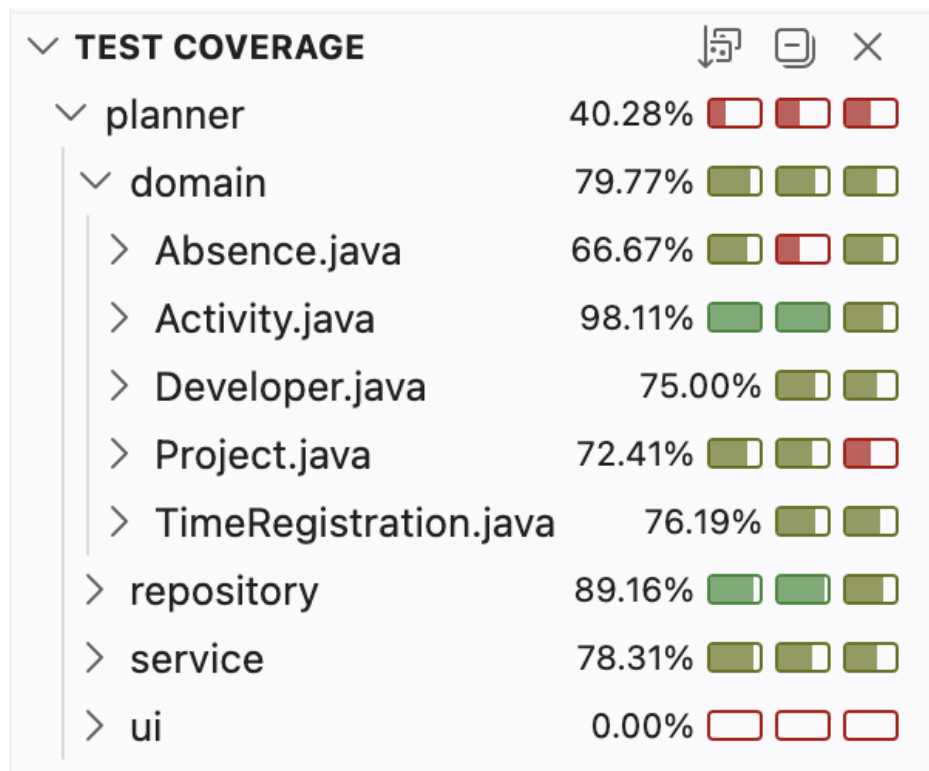


Figure 4: Per-class coverage within `com.planner.service`.

5.3.2 Domain layer 79.77%

Domain objects are thin data holders whose behaviour is exercised indirectly through the service tests. Figure 5 shows the per-class breakdown. `Activity.java` reaches 98.11% because activities are manipulated in almost every use case, while `Absence.java` stops at 66.67%, reflecting the smaller number of absence-related scenarios. The gap from 100% consists entirely of trivial getters, `toString`, `getType`, `setDate` that no test calls directly; the project brief explicitly advises against testing bare getters.

Figure 5: Per-class coverage within `com.planner.domain`.

5.4 Effect of the White-Box Tests

The Cucumber scenarios verify business behaviour from the outside: they describe happy paths and the most prominent error cases in natural language, and they exercise whole call chains from the service layer down into the domain. Because they are written in terms of business outcomes, they are not designed to target individual branches or boundary values inside a method.

The 25 JUnit tests introduced in Section 3 fill exactly this gap. Using equivalence partitioning and boundary value analysis, they target the code's internal control-flow structure: the boundary values 0, 1, 53 and 54 in week-number validation; the three separate null/empty/whitespace cases for project names; and the conflict check between a new absence and an existing activity assignment. These are branches the Cucumber suite rarely or never reaches.

The combined suite produces a visibly tighter branch coverage on `ProjectService` and `ActivityService` than either approach achieves alone. This is not a coincidence; it is the empirical confirmation of the standard software engineering claim that black box specification-based testing and white box structural testing are complementary methodologies, each reaching parts of the code the other leaves untouched.

5.5 Limitations

Coverage measures *execution*, not *correctness*. A test that calls a method without asserting anything contributes fully to coverage while verifying nothing. The figures in Table 11 should therefore be read alongside the Cucumber scenarios, behavioural correctness, the JUnit assertions (boundary correctness), and the design-by-contract checks in Section 4. Taken together, these four sources of evidence provide a considerably stronger argument for test adequacy than coverage alone.

5.6 Summary

Table 12 provides a compact overview of the main findings.

Layer	Coverage	Interpretation
Service	High (78.31%)	TDD followed; white-box tests close remaining branch gaps
Domain	Medium (79.77%)	Thin data classes; uncovered code is trivial getters only
Repository	High (89.16%)	Reached transitively; gap is interface bytecode artefact
UI	None (0.00%)	Deliberately excluded; business logic tested at service level

Table 12: Summary of coverage findings per architectural layer.

6 Design Patterns and SOLID Principles (Nicolai)

6.1 Overall Design Choices

This project is centered around a layered architecture, where the code is separated into clear and separate *domain-classes*, *services*, *repositories*, and *user interface* as illustrated in 6

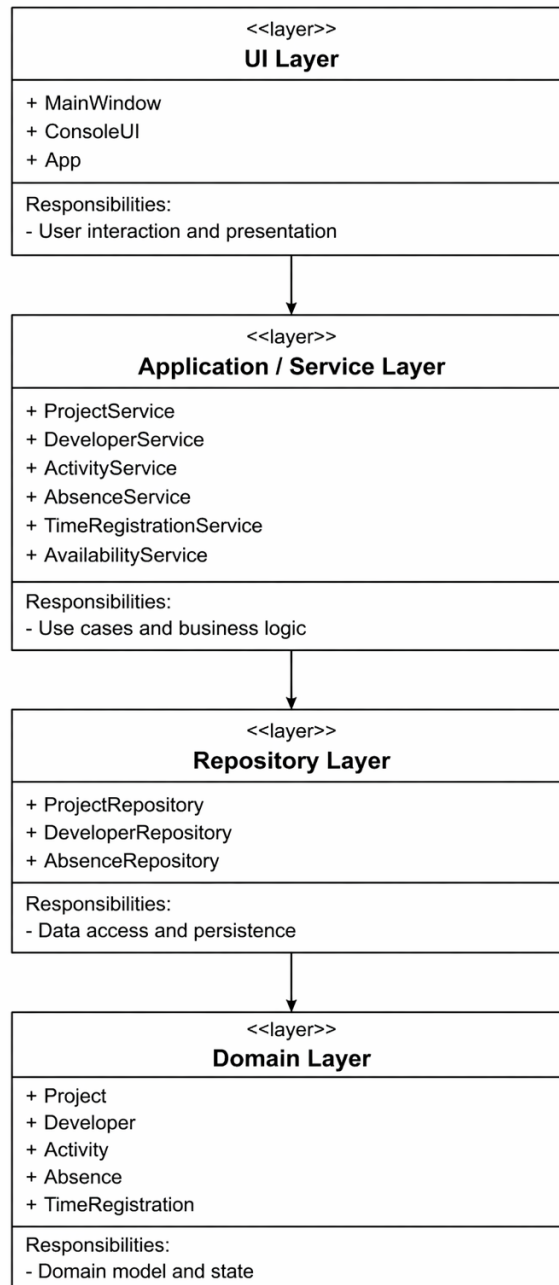


Figure 6: The layered achitecture of the system

This was done to create clear separation between files to make it possible for the group to work in parallel in tandem without too many dependencies between parts. Clarity, separation and function-independence has been in the groups design-scope from the start.

6.1.1 Domain Layer

The domain-layer carries the central objects of the program and includes the domain-files *Project*, *Activity*, *Developer*, *Absence* and *TimeRegistration*. These classes represent all data and relations within the system, and the program as a whole depends solely on them for the overall state of the system. User interaction, storage and higher application logic such as system rules, are kept out of this layer, to minimize dependencies to other parts of the system and avoid coupling between layers. For example, the *Project* class contains basic project information and a relation to its activities:

```

1      public class Project {
2          private final String id;
3          private final String name;
4          private Developer projectLeader;
5          private final List<Activity> activities = new ArrayList<>();
6
7          public Project(String id, String name) {
8              this.id = id;
9              this.name = name;
10         }
11     }

```

This way, the domain classes only store information about the given projects, and the relation to other domain objects, keeping presentation, general data storage and system rules out of the domain layer. These functions and responsibilities are handled by the service, repository and interface layers.

6.1.2 Repository Layer

The repository-layer is used to isolate general data storage from the rest of the system. The classes, *ProjectRepository*, *DeveloperRepository* and *AbsenceRepository* are in charge of saving and gathering data from the other layers in the system. In final iterations of the project, in-memory storage was introduced into these classes, keeping objects inside the repositories for easier testing. For an example, in the project repository

```

1      public class ProjectRepository {
2          private final List<Project> projects = new ArrayList<>();
3
4          public void add(Project project) {
5              projects.add(project);
6          }
7      }

```

```

8      public Project findById(String id) {
9          // searches the list of projects
10     }
11 }

```

This way data is kept inside the single program as seen in 7

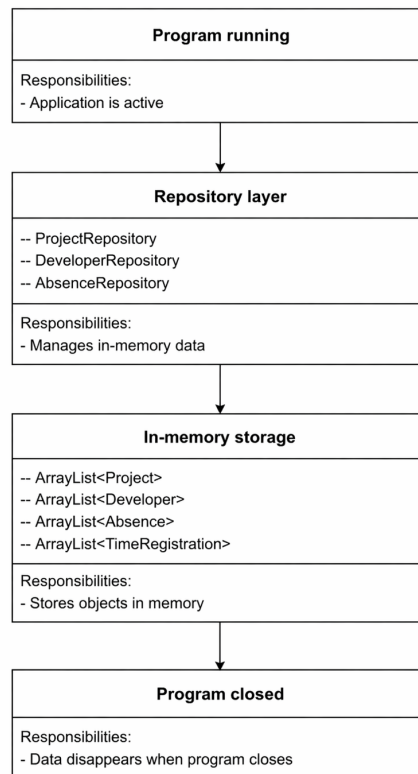


Figure 7: In-memory repositories

This was done as a deliberate design choice to support system testing as well as application logic for the group and further development, rather than having a persistent storage introducing unnecessary complexity for the required functionality of the program.

6.1.3 Service layer

While the state of the system is upheld and kept by the domain-layers, and data storage is kept by the repositories, the application logic is solely placed in the systems service layers. The layer, consisting of the classes *ProjectService*, *ActivityService*, *TimeRegistrationService*, and *AbsenceService*, each handle their own specific part of the systems functionality. This layer is responsible for coordinating all operations within the project, enforcing system rules, and keeping developers, leaders and projects aligned with each other. For example, the creation of an activity is handled in the *ActivityService*:

Listing 3: Activity creation in service layer

```

1 public void createActivity(String projectId, String activityName) {
2     Project project = projectRepository.findById(projectId);
3     // validation and logic
4     project.addActivity(new Activity(activityName));
5 }

```

As seen in the listing above, the *ActivityService* is end the *projectId* and *ActivityName*, then finds the correct project in the repository, validates and applies the necessary logic needed. The domain object itself does not handle how activities are created, and the repository is only responsible for storing and retrieving data.

6.1.4 User Interface Layer

The user interface layer is the interaction between the program and end user. This interface consist of a console-based interface and a graphical JavaFX-based interface. The UI layer is where all before mentioned logic is used or presented to the user. User input, and program functionality happens at this level, while keeping the actual logic hidden. Instead input is delegated to the before mentioned services, *ProjectService*, *ActivityService*, and *AbsenceService*. For an example, if a user wants to create a new project through the interface, the UI collects the input and call the relevant service-layer method needed. Validation, System logic and updates are then kept in these layers, leaving the interface layer as prestation and input only.

```

1 projectService.createProject(projectName);

```

This way, all system logic is kept from the end user, all while keeping all necessary information viewable in the *MainWindow*. It should be noted this is the overall politics of the interface, as there is quite an amount of UI-related code viewable instead of it being a full out developed GUI architecture. This was due to it being one of the last features developed, and was agreed upon as acceptable trade-off for focus on the project service and domain logic.

6.1.5 Overall Design Considerations

It should be noted that the layered architecture was choosen because it was the best way to balance between structure, clarity and simplicity. While more advanced architectural patterns could have been applied, this approach was considered sufficient for the scope of the project.

The design prioritizes ease of development, making the system easier to test, maintain and extend without introducing unnecessary complexity.

6.2 SOLID-principles

In this project, focus has been on a structured system with clarity, testability, and further development in mind. As described in the layered architecture above, the system is already separated into *domain-classes*, *services*, *repositories*, and *user interface*, which naturally

aligns with several SOLID principles.

We have used the SOLID-principles to the extent we found useful for a project of this size and scope. Readability and understanding of the code has been prioritized, rather than forcing design patterns where they did not solve a specific problem.

The layered architecture, repository pattern, and dependency injection are among the design choices made, while principles such as SRP and DIP are the most prominent in the final system.

6.3 Single Responsible Principle

The program is highly structured around the Single Responsibility Principle through the layered design described earlier. Each part of the system has a clear primary responsibility, which contributes to a more clear and maintainable codebase.

As illustrated in Figure 6, the separation between the layers is prominent, where each layer is responsible for a distinct part of the system, and the functionality is distributed accordingly.

6.3.1 Domain-classes

The layered structure and separation is easy to spot in the projects classes. Looking into the domain-classes, *Project*, *Activity*, *Developer*, *Absence* and *TimeRegistration*, we see that they primarily represents the relations and data within the system. Together, they keep system truth aligned and ensures the consistency of the data. They maintain the state that the program actually works within, making the project work.

For example, a *Project* contains a list of activities and basic project information:

```

1  public class Project {
2      private String name;
3      private Developer projectLeader;
4      private List<Activity> activities;
5  }
```

On the other hand, these classes are deliberately kept independent from the end user and data storage responsibilities. Data storage is instead handled by the repository layer.

6.3.2 Repositories

The repository layer is divided into classes such as *ProjectRepository*, *DeveloperRepository* and *AbsenceRepository*, each managing their own specific part of system-data. For an instance, the *projectRepository* is responsible for storing and retrieving the different projects inside the system:

```

1  public class ProjectRepository {
2      private List<Project> projects = new ArrayList<>();
```

```

3
4     public void add(Project project) {
5         projects.add(project);
6     }
7
8     public Project findById(String id) {
9         // lookup logic
10    }
11 }

```

Separation between the two layers, *domain and repositories*, ensures that changes to data storage does not directly affect the logic of the domain logic. This does not only make the system more manageable, but also flexible for further development if needed.

6.3.3 Service layer

The service layer coordinates operations between the domain and repository layers, while enforcing system rules and logic.

Through the layers just described, *Domain, Repository and Service*, it is clearly shown how responsibilities are separated. This directly reflects the Single Responsibility Principle, where each layer has a clear and limited role in the overall system.

6.4 Dependency Inversion Principle

During late development of the final version of the project, the service layer was improved even further by applying repository interfaces such as *IProjectRepository*, *IAbsenceRepository* and *IDeveloperRepository*, using another SOLID principle, Dependency Inversion Principle. This was done to improve the overall design and future maintainability, with these introduced interfaces, service classes, *ProjectService*, *AbsenceService* and *DeveloperService*, no longer depend on the repository implementation, but use the *I-repositories* as abstractions. An example of this, can be seen in the constructor in the three serviceclasses, here *ProjectService*

```

1     public class ProjectService {
2         private final IProjectRepository projectRepository;
3
4         public ProjectService(IProjectRepository projectRepository) {
5             this.projectRepository = projectRepository;
6         }
7     }

```

This design choice, with DIP in mind, reduced the general dependency the service and storage had on each other. With this alteration, the general repository implementation happens outside the class, instead of being created internally. This makes the design more flexible in further development, because it makes it possible to replace general repository implementations without the need for change in the service logic itself.

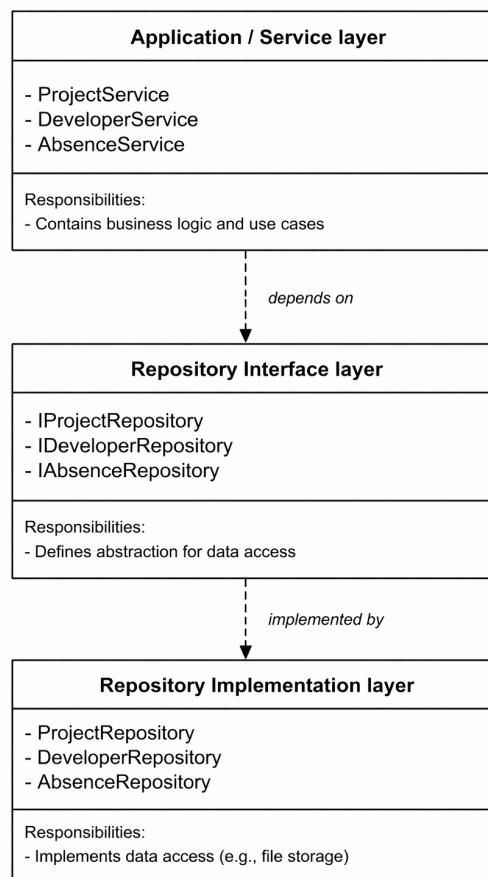


Figure 8: DIP-flow (skal renames er træt)

6.5 Other SOLID-Principles

Single Responsibility and Dependency Inversion, are by design the most prominent and envisioned principles used in the Project. The remaining principles in SOLID, *Open/closed*, *Interface Segregation* and *Liskov* are only partially used in the design.

6.5.1 OCP, Open/Close principle

The Open/close principle, which is about the code being open for extension, but closed to modification, is only partially used in the service layer. Here, new functionality can be coded through service methods without actually modifying the domain classes directly.

```

1  public class ProjectService {
2
3  public void createProject(String name) {
4      // existing functionality
5  }
6
7  // New functionality
8  public void calculateProgress(String projectId) {
9      Project project = projectRepository.findById(projectId);
10     -> //new logic added without modifying Project class
11 }
12 }
```

This only points in the direction of real OCP, because of the need to change the service class to add functionalities to the code, and it can be concluded that the system is not designed around OCP extensibility.

The scope the group have had throughout the coding process has been on clarity, testability, and well structured understandable code that supports the structured implementation of the core functionality. To fully implement and support the open/close principle, would require a design overhaul, and result in complexity which we did not consider necessary for the overall purpose and functionality of the system we already build.

6.5.2 Interface Segregation Principle

The Interface Segregation states, that clients should not depend on functionality they do not need. Instead, the program should separate out the needed functionality in interfaces. In our program, ISP is not that prevalent. We did not feel the need to split the functionality up into separate interfaces more than it already is. Every client, the developer or project leader, have a very little interface already to begin with. The only place where ISP is somewhat used is again in the before mentioned repository interfaces, Separate out needed functionality in interfaces, *IProjectRepository*, *IAbsenseRepository* and *IDeveloperRepository*. However, this structure primarily follows the overall repository design, rather than being a direct result of applying ISP as a design principle.

6.5.3 Liskov Substitution Principle

At last the Liskov Substitution Principle, which states that subclasses should be changeable for their general base classes without affecting the functionality and correctness of the overall program. Or in simpler terms, classes should be able to inherit from other subclasses without change in the code. In our project, LSP is not used in the design. The system we made do not rely on inheritance of class hierarchies, but instead primarily focuses on composition and clear separation between the layers. As a result there are very few functions where substitution between subclasses are relevant, and the Liskov Substitution Principle are therefore not applied in the system intentionally. This reflects the overall design choice the group made to begin with, where composition and clarity was in a higher priority.

6.5.4 Final remarks

Overall, the use of SOLID principles in this project has been pragmatic rather than strict. The focus has been on creating a clear and maintainable structure, where the most relevant principles, such as SRP and DIP, have been applied where they added value. Other principles have only been partially used or not applied, as the added complexity was not considered necessary for the scope and functionality of the system.

7 Conclusion

The final system implements the central functionality described in the project requirements. It supports creating projects and activities, assigning developers, registering time, registering absence, assigning project leaders, viewing available developers, and generating project reports. These features cover the most important parts of the project planning workflow and show that the final implementation follows the core direction established in Report 1.

The development process also showed that the design from Report 1 had to be made more concrete during implementation. Some parts of the system were simplified to fit the project scope, especially persistence, which is handled with in-memory repositories. However, the final architecture still reflects the original design idea: domain classes represent the central concepts, repositories handle storage, services contain business logic, and the user interface delegates work to the service layer. This separation made the implementation easier to test and reason about.

The testing work gave a more precise understanding of the system's behaviour. The BDD scenarios verify that the system supports the main use cases from a user perspective, while the white-box tests focus on selected service methods and their internal decision points. This combination is useful because it checks both requirement-level behaviour and method-level correctness. The Design by Contract section strengthens this further by expressing the expected behaviour of the same selected methods as preconditions and postconditions.

The code coverage results show that the service, repository, and domain layers are the most relevant parts of the system to test, because they contain the business rules and state changes. The user interface has lower coverage, which is expected because the GUI was not developed using the same test-driven approach as the service layer. This is a limitation of the project, but it is also a deliberate prioritisation: the core functionality was tested where the risk of logical errors is highest.

Overall, the project resulted in a working and structured implementation of a software project planner. The final system is not a complete production system, since it does not include persistent database storage or advanced user management, but it demonstrates the required functionality and a clear layered architecture. Compared with Report 1, the final product is more implementation-focused and testable, while still preserving the original goal of supporting project planning, resource allocation, and progress tracking.

The main lesson from the project is that a clear separation between domain logic, repositories, services, and user interface makes the system easier to develop, test, and document. The connection between use cases, white-box tests, contracts, and coverage also gives a stronger basis for evaluating whether the implementation actually satisfies the intended behaviour.

7.1 Reflection on the Development Process

The project was developed collaboratively across the group. Work was divided across analysis, implementation, testing, and documentation tasks, while the final solution was continuously adjusted as the implementation became more concrete. This meant that the design from Report 1 functioned as a starting point, but some details had to be refined when the actual Java classes, service methods, repositories, and user interface were implemented.

One important part of the process was the separation of responsibilities in the code. By dividing the system into domain classes, repositories, services, and user interface classes, the group could work on different parts of the system without everything depending on one large class. This also made the system easier to test, because the service layer contains most of the business logic and can be tested independently from the JavaFX interface.

The testing work also influenced the development process. The BDD scenarios helped connect the implementation to the use cases, while the white-box tests gave a more detailed view of specific methods and their decision points. This made it easier to identify which parts of the implementation needed clear validation logic and which methods were most important to document with Design by Contract.

Compared with Report 1, the final system is more implementation-oriented and concrete. Some features were simplified to keep the project manageable, such as using in-memory repositories instead of database persistence. However, the central purpose of the system remained the same: supporting project planning, developer assignment, time registration, absence registration, and project progress tracking. Overall, the process showed how the initial design can guide development, while still needing practical adjustments during implementation. Use of AI

8 Collaboration and Contribution Weights

Table 13 shows the contribution weights for each group member.

s240803	s234146	s245203	s245010	s240335
1.0	1.0	1.0	1.0	1.0

Table 13: Contribution weights for the group members

9 Appendix

9.1 Git Repository

The Git repository for the project is available at:

[Insert repository URL here]

9.2 Run Instructions

After unpacking the ZIP archive, the project can be imported as a Maven project in Eclipse, IntelliJ IDEA, or Visual Studio Code.

Run application

```
mvn javafx:run
```

Run tests

```
mvn test
```

The system includes at least one developer with the initials `huba`, as required in the project description.

List of Figures

1	VS Code <code>test coverage</code> panel at package level after executing all 57 tests. .	25
2	Per-class coverage within <code>com.planner.repository</code>	25
3	Per-class coverage within <code>com.planner.ui</code>	26
4	Per-class coverage within <code>com.planner.service</code>	27
5	Per-class coverage within <code>com.planner.domain</code>	28
6	The layered architecture of the system	30
7	In-memory repositories	32
8	DIP-flow (skal renames er træt)	36

List of Tables

1	Input properties for <code>createProject(...)</code>	6
2	Concrete test cases for <code>createProject(...)</code>	6
3	Input properties for <code>setActivityDetails(...)</code>	8
4	Concrete test cases for <code>setActivityDetails(...)</code>	8
5	Input properties for <code>addDeveloperToActivity(...)</code>	9
6	Concrete test cases for <code>addDeveloperToActivity(...)</code>	10
7	Input properties for <code>registerTime(...)</code>	12
8	Concrete test cases for <code>registerTime(...)</code>	12
9	Input properties for <code>registerAbsence(...)</code>	13
10	Concrete test cases for <code>registerAbsence(...)</code>	13
11	Line coverage per package as reported by JaCoCo via VS Code's <code>test coverage</code> panel.	24
12	Summary of coverage findings per architectural layer.	29
13	Contribution weights for the group members	40

References